МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики

Спектральная кластеризация в задаче сегментации изображений

Исследовательский проект
студента 202 группы факультета ВМК МГУ
Грознецкого Арсения Евгеньевича

Москва
2024

Содержание

Введение **2**
Постановка задачи . . . 2
Краткое описание метода решения . . . 2

Преобразование изображения в граф **3**
Правило преобразования . . . 3

Спектральная теория графов **4**
Основные понятия и утверждения . . . 4
Задача max-flow min-cut . . . 5
Теория Фидлера . . . 5

Собственные пары разреженных матриц **6**
Степенной метод . . . 6
Обратный степенной метод . . . 6
Метод минимальных невязок . . . 7

Реализация и тестирование алгоритма **8**
Выделение объекта . . . 8
Выделение границ . . . 10
Алгоритмическая сложность . . . 12

Заключение **14**
Плюсы и минусы алгоритма . . . 14
Дальнейшее развитие . . . 14

Список литературы **15**

Введение

Постановка задачи

Сегментация изображений — задача о нахождении множества пикселей, отвечающих объекту на изображении — бесспорно является акутальной и с развитием нейронных сетей приобретает новые эффективные решения. Другой подход: алгоритмическое выделение объекта — удобен для вычислений на локальном компьютере, и может найти применение и развитие в других сферах. В данной работе автор решает задачу сегментации, используя спектральную теорию графов и численные методы линейной алгебры.

Краткое описание метода решения

Точные формулировки понятий будут даны в основной части работы.

Входные данные: изображение **Image** размера $w \times h$. При решении задачи изображение преобразуется в *граф* $\boldsymbol{G}$: $|\boldsymbol{G}| = n = w \cdot h$. Вершины графа $\boldsymbol{G}$ соответствуют пикселям изображения, а ребра характеризуют разницу цвета каждого пикселя с соседними 8-ю (по вертикали, горизонтали и двум диагоналям). По графу $\boldsymbol{G}$ строится матрица смежности и *лапласиан* $\mathbf{L} \in \mathbb{Z}^{n \times n}$, который дает достаточную для сегментации информацию. Выделение объекта на изображении происходит с помощью *вектора Фидлера* $\mathbf{f} \in \mathbb{R}^n$— собственного вектора лапласиана $\mathbf{L}$, отвечающего второму (следующему после минимального) собственному значению $\mathbf{L}$.

Таким образом, задача о выделении объекта сведена к поиску второй собственной пары, однако в работе показано, что вместо нее необходимо искать минимальную с ненулевым собственным значением. По построению матрица $\mathbf{L}$ разрежена, и с помощью обратного степенного метода и метода минмиальных невязок можно легко найти минимальное ненулевое собственное значение и собственный вектор

Дальнейшие действия по выделению объекта на изображении основываются на свойстве ветора Фидлера: при упорядочивании его компонент по возрастанию возникают положительные и отрицательные элементы. Разбиение вершин в соответствии со знаком элемента в векторе Фидлера дает разбиение графа на две области с лучшей оценкой (с точки зрения *проводимости* графа $\boldsymbol{G}$).

Выходные данные: множество пикселей, отвечающих объекту на изображении, и дополнительное множество, отвечающее заднему фону.

Преобразование изображения в граф

Для изучения свойств изображения **Image** размером $w \times h$ сопоставим изображению **Image** *граф* $\boldsymbol{G}$.
Граф $\boldsymbol{G} = \langle V, E \rangle$: $V \neq \emptyset$, $E \subseteq V \times V$.

Правило преобразования

В качестве V выбирается множество пикселей **Image** ($|V| = n = wh$), "вытянутое" по строкам изображения. Последнее формально значит:

$$v_k = \mathbf{Image}[k//w, k\%w], \forall v_k \in V$$

Операции // и % обозначают целочисленное деление и взятие остатка.

В качестве множества ребер $e \in E$ рассматривается множество пар соседних пикселей **Image**. И в зависимости от цветовых различей соседних пикселей ребро $e \in E$, либо $e \notin E$. Цветовое различие определяется как разность серых компонент пикселей. Получить матрицу с серыми компонентами каждого пикселя из изображения можно программно с помощьюю средств библиотеки `skimage` языка `Python`. Если модуль разности преодолевает пороговое значение, то цвета различны и $e \notin E$, иначе — цвета близки и $e \in E$.

Таким образом, построенный граф $\boldsymbol{G}$ имеет структуру слабосвязанных между собой *областей связности* — областей с большим числом ребер внутри — отвечающих объектам изображения, выделенным по цветовому признаку.

Спектральная теория графов

Основные понятия и утверждения

Для дальнейшего изучения изображения построим по графу $\boldsymbol{G}$ — *матрицу смежности* $\mathbf{A}$.

Матрица смежности $\mathbf{A} \in \mathbb{Z}^{n\times n}$ графа $\boldsymbol{G}$, $|\boldsymbol{G}| = n$:

$$a_{ij} = \begin{cases} 1, & \text{если}(i,j) \in E, \\ 0, & \text{иначе} \end{cases}$$

Очевидно: если граф неориентированный и без петель, то $\mathbf{A}$ симметричная с нулевыми диагональными элементами. По матрице смежности определяется *лапласиан* графа $\boldsymbol{G}$.

Лапласиан $\mathbf{L} = \mathbf{D} - \mathbf{A}$. Здесь $\mathbf{D} = \mathrm{diag}(\deg(v_1)...\deg(v_n))$,
где $\deg(v_i)$— число исходящих ребер.

Для дальнейшей работы необходимо привести несолько тривиальных утверждений, которые можно проверить непосредственно.

Лемма:
$\forall \mathbf{x} \in \mathbb{R}^n : \mathbf{x}^T\mathbf{L}\mathbf{x} = \sum\limits_{(i,j)\in E} (x_i - x_j)^2$.

Утверждение 1:
$\mathbf{L}$ — неотрицательно определенная.

Следствие:
Все собственные значения $\mathbf{L}$ неотрицательны: $0 \le \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$.

Утверждение 2:
$\mathbf{1} = (1, 1, \ldots, 1)^T$ и 0 — собственная пара $\mathbf{L}$.

Утверждение 3:
Если граф $\boldsymbol{G}$ имеет две компоненты связности, то геометрическая кратность нулевого собственного значения равна двум.

Обоснование:
При построении лапласиана пронумеруем $1, ..., k$ вершины из первой компоненты, и $k+1, ..., n$ — вершины из второй компоненты. Тогда лапласиан представляет собой блочную диагональную матрицу, и к каждому блоку можно применить *Утверждение 2*.

Следствие:
Геометрическая кратность нулевого собственного значения равна числу компонент свзяности графа.

Задача max-flow min-cut

Выделение областей связности графа $\boldsymbol{G}$ непосредственно связано с решением задачи о нахождении *минимального сечения с максимальным потоком* (max-flow min-cut). Для постановки задачи формализуем эти понятия, а именно рассмотрим понятие *проводимости.*

Проводимость множества $S \subseteq V$ есть величина:

$$\varphi(S) = \frac{|E(S, S^c)|}{\min(\mathrm{vol}(S), \mathrm{vol}(S^c))}$$

где:
$S^c = V \backslash S$;
$E(A, B) = \{(i, j) \in E : i \in A, j \in B\}$, где $A, B \subseteq V$;
$\mathrm{vol}(A) = \sum\limits_{i \in A} \deg(i)$, где $A \subseteq V$.

Комментарий:
В выражении выше: минимизация числителя по S – задача о поиске min-cut, максимизация знаменателя — max-flow. Решение задачи min-cut помогает разбить граф на две слабосвязанных между собой области, то есть отделить объект на изображении от фона — это можно сделать с помощью теории Фидлера.

Теория Фидлера

Мирослав Фидлер (1926 – 2015 гг.) — чешский математик — известен своим вкладом в спектральную теорию графов [1].

Вектором Фидлера $\mathbf{f} \in \mathbb{R}^n$ называется собственный вектор матрицы $\mathbf{L}$:
$\mathbf{Lf} = \lambda_2 \mathbf{f}$.

Теорема (Фидлер):
Если $\mathbf{f}$ — вектор Фидлера, то множество вершин графа $\boldsymbol{G}$, отвечающее положительным элементам ветора, является решением задачи min cut. То есть, если вектор $\mathbf{f}$: $\mathrm{f}_{i_1} \leq \mathrm{f}_{i_2} \leq \cdots \leq \mathrm{f}_{i_k} \leq 0 < \mathrm{f}_{i_{k+1}} \leq \cdots \leq \mathrm{f}_{i_n}$, то:
$S_0 = \{v_{i_1}, v_{i_2}, ..., v_{i_k}\}$ — есть решение задачи min cut [2].

Применение:
Граф $\boldsymbol{G}$ состоит из нескольких компонент связности (как минимум две: объект и фон). Вместе с этим граф $\boldsymbol{G}$ может содержать компоненты связности только из одной вершины, если пиксель, отвечающий этой вершине, “выбивается” по цвету. Последний факт затрудняет использование теоремы Фидлера для выделения объекта, так как второе минимальное собственное значение в силу *Утверждения 3* также нулевое, и вместо выделения объекта будет выделен “выбивающийся” пиксель. Поэтому для выделения объекта вместо вектора Фидлера будет использоваться собственный вектор, отвечающий **минимальному ненулевому** собственному значению, этот вектор даст приближенное решение задачи min-cut. То есть, задача сведена к поиску собственного ветора.

Собственные пары разреженных матриц

По построению $\mathbf{L}$ в каждой строке не больше 9 отличных от нуля элементов. То есть матрица $\mathbf{L}$ разрежена, что позволяет существенно уменьшить количество арифметических операций при матричных умножениях. Численные методы для вычисления минимального отличного от нуля собственного значения и соответствующего собтсвенного вектора взяты из [3].

Степенной метод

Степенной метод позволяет находить наибольшую собственную пару — решение этой задачи легко сводится к нахождению наименьшей ненулевой собственной пары. Поэтому рассмотрим данный алгоритм.

Алгоритм: Степенной метод

1. $\mathbf{v}^{(0)}$ — случайный вектор с $||\mathbf{v}^{(0)}||_2 = 1$
2. **for** $k = 1, 2, 3, ...$
   $\mathbf{w} = \mathbf{L}\mathbf{v}^{(k-1)}$
   $\mathbf{v}^{(k)} = \mathbf{w}/||\mathbf{w}||_2$

Обоснование: Предположим, что $\mathbf{L} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ — диагонализуема с $||\mathbf{v}_1||_2 = 1$ и $\mathbf{\Lambda} = \mathrm{diag}(\lambda_m, \lambda_{m-1}, ..., \lambda_1)$.
Пусть $(\alpha_1, \alpha_2, ..., \alpha_m)^T = \mathbf{V}^{-1}\mathbf{v}^{(0)}$. Если $\alpha_1 \neq 0$, тогда:

$$\mathbf{L}^k\mathbf{v}^{(0)} = \mathbf{V}\mathbf{\Lambda}^k\mathbf{V}^{-1}\mathbf{v}^{(0)} = \mathbf{V}\begin{bmatrix} \alpha_1\lambda_m^k \\ \alpha_2\lambda_{m-1}^k \\ ... \\ \alpha_m\lambda_1^k \end{bmatrix} = \alpha_1\lambda_m^k\mathbf{V}\begin{bmatrix} 1 \\ \frac{\alpha_2}{\alpha_1}(\frac{\lambda_{m-1}}{\lambda_m})^k \\ ... \\ \frac{\alpha_m}{\alpha_1}(\frac{\lambda_1}{\lambda_m})^k \end{bmatrix}$$

Таким образом, $\mathbf{v}^{(k)} = \frac{\mathbf{L}^k\mathbf{v}^{(0)}}{||\mathbf{L}^k\mathbf{v}^{(0)}||_2}$, и $e^{-i\theta_k}\mathbf{v}^{(k)} \longrightarrow \mathbf{v_1}$, где $\theta_k = k\theta + \theta_0$, такое что: $e^{i\theta} = \lambda_m/|\lambda_m|$ и $e^{i\theta_0} = \alpha_1/|\alpha_1|$.

В качестве критерия остановки алгоритма можно рассмотреть условие:

$$||\mathbf{v}^{(k)} - \mathbf{v}^{(k-1)}||_2 < \epsilon$$

Обратный степенной метод

Обратный степенной метод позволяет находить наименьшую ненулевую собственную пару, и он очень похож на степенной метод. Основывается он на свойстве обратной матрицы: собственные векторы обратной матрицы совпадают с собственными векторами исходной матрицы. Нетрудно видеть:

$$\mathbf{L}\mathbf{v} = \lambda\mathbf{v} \Longleftrightarrow \mathbf{L}^{-1}\mathbf{v} = \lambda^{-1}\mathbf{v}$$

Поэтому наименьшее собственное значение матрицы $\mathbf{L}$ отвечает наибольшему собственному значению матрицы $\mathbf{L}^{-1}$. И применяя **степенной метод** к матрице $\mathbf{L}^{-1}$, мы получим искомый вектор: отвечающий наименьшему ненулевому собственному значению $\mathbf{L}$. Однако в этом месте возникает новая трудность: на каждой итерации необходимо вычислять вектор $\mathbf{w} = \mathbf{L}^{-1}\mathbf{v}$. О том, как это делать, не вычисляя $\mathbf{L}^{-1}$, речь пойдёт дальше.

Метод минимальных невязок

Задача о нахождении вектора $\mathbf{w}$: $\mathbf{w} = \mathbf{L}^{-1}\mathbf{v}$ эквивалентна задаче о решении системы $\mathbf{Lw} = \mathbf{v}$. Для её решения воспользуемся **методом минимальных невязок**, предполагая совместность системы.

Последнее утверждение оправдано тем, что событие {система несовместна} есть событие {начальный случайный ветор $\mathbf{v}^{(0)}$ не лежит в образе $\mathbf{L}$}, то есть в разложении $\mathbf{v}^{(0)}$ по базису образа $\mathbf{L}$ и базису дополнительного пространства есть компоненты из обоих пространств. В силу *Утверждения 3* размерность ядра $\mathbf{L}$ равна числу компонент связности графа, которое мало по построению графа (компоненты: фон, объект, "выбивающиеся" по цвету пиксели) и которым можно пренебречь по сравнению с порядком матрицы $\mathbf{L}$. То есть, вероятность события {система несовместна} — мала. Перейдем к методу:

Невязкой называется вектор $\mathbf{r} = \mathbf{v} - \mathbf{Lw}$. Очевидно, что если подставить решение системы $\mathbf{Lw} = \mathbf{v}$, то невязка станет нулевой. Таким образом минимизация невязки эквивалентна задаче о решении системы. Будем минимизировать $(\mathbf{r}, \mathbf{r})$.

Итерационная формула:
$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \tau_k \mathbf{r}^{(k)}$

Рассмотрим разность невязок $\mathbf{r}^{(k+1)}$ и $\mathbf{r}^{(k)}$:
$\mathbf{r}^{(k+1)} - \mathbf{r}^{(k)} = \mathbf{Lx}^{(k)} - \mathbf{Lx}^{(k+1)} = \mathbf{Lx}^{(k)} - \mathbf{L}(\mathbf{x}^{(k)} - \tau_k \mathbf{r}^{(k)}) = \tau_k \mathbf{Lr}^{(k)}$

Откуда:
$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} + \tau_k \mathbf{Lr}^{(k)}$

Минимизируем по τ_k невязку $\mathbf{r}^{(k+1)}$:
$(\mathbf{r}^{(k+1)}, \mathbf{r}^{(k+1)}) = (\mathbf{r}^{(k)}, \mathbf{r}^{(k)}) + 2\tau_k(\mathbf{Lr}^{(k)}, \mathbf{r}^{(k)}) + \tau_k^2(\mathbf{Lr}^{(k)}, \mathbf{Lr}^{(k)})$

Откуда минимум достигается при:
$\tau_k = -\frac{(\mathbf{Lr}^{(k)}, \mathbf{r}^{(k)})}{(\mathbf{Lr}^{(k)}, \mathbf{Lr}^{(k)})}$

Пересчитывая на каждой итерации τ_k, $\mathbf{x}^{(k)}$ и $\mathbf{r}^{(k)}$, получим алгоритм для вычисления решения. Критерием остановки является достижение достаточной близости к нулю второй нормы вектора невязки.

Об оценках сходимости каждого численного метода можно прочить в [3].

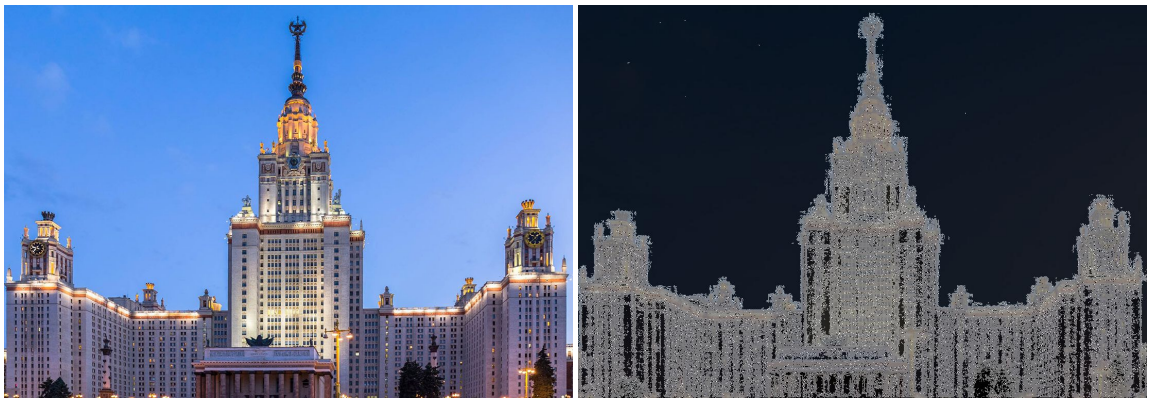
Реализация и тестирование алгоритма

Разработанный алгоритм сегментации изображений был реализован на языке `Python` с использованием библиотек `numpy` и `scipy`. Код имеет структуру:

- преобразование изображения в лапласиан
- вычисление искомого собственного вектора
- построение множества точек объекта

Выделение объекта

Результат выполнения программы на различных изображениях можно видеть ниже:



(a) original (b) detected object

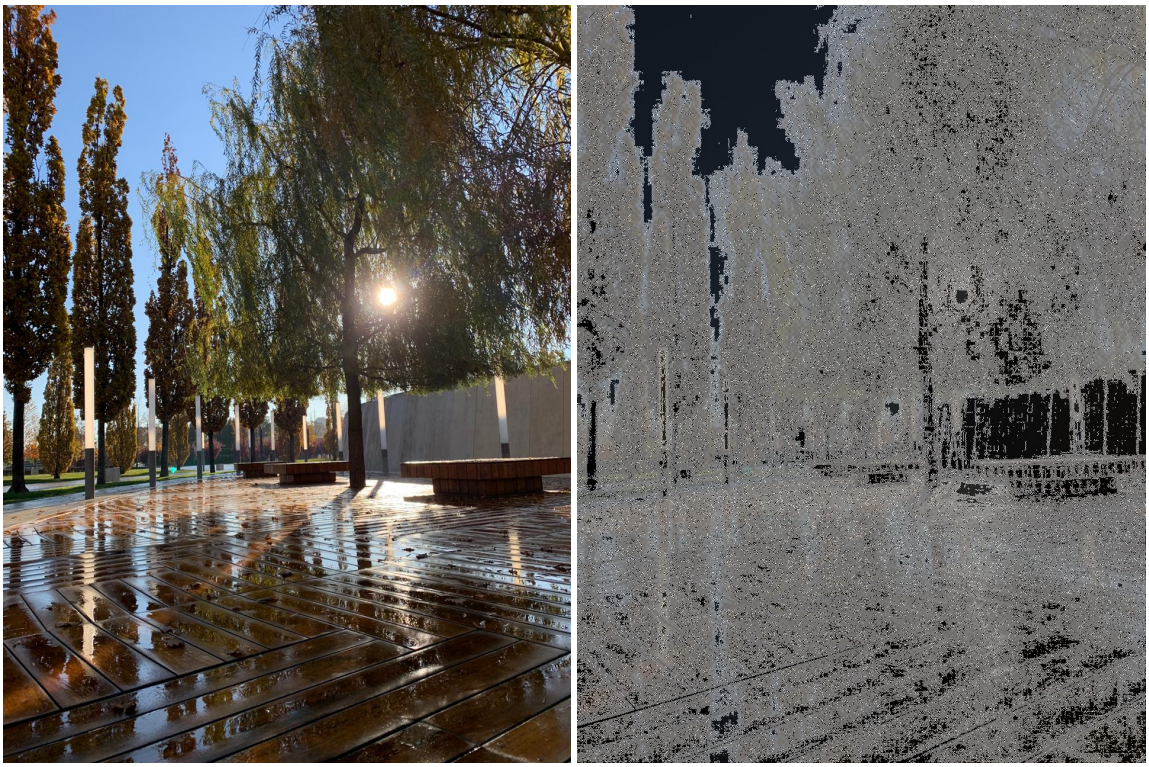
Рис. 1: MSU.

(a) original (b) detected object

Рис. 2: Cat.

(a) original (b) detected object

Рис. 3: Apple.



(a) original (b) detected object

Рис. 4: Trees.

Также данный алгоритм можно применить для выделения границ объектов на изображении, если поменять логику “пробрасывания” ребра между пикселями на обратную: ребро есть, если пиксели различных цветов, иначе — ребро отсутствует. При такой логике большим числом связей в графе обладают границы, и с помощью теории Фидлера будут выделены именно они. Примеры применения приведены в разделе ниже.

Выделение границ

Результат выполнения алгоритма с модификацией для выделения границ:

(a) original (b) detected edges

Рис. 5: Apple edges.

(a) original (b) detected edges

Рис. 6: Orchestra edges.

(a) original (b) detected edges

Рис. 7: Words edges.

(a) original

(b) detected edges

Рис. 8: Moscow-city edges.

(a) original

(b) detected edges

Рис. 9: MSU edges.

(a) original (b) detected edges

Рис. 10: Sunset edges.

Алгоритмическая сложность

Рассмотрим вопрос сложности алгоритма. У описанного алгоритма два вычислительных ядра, на которые приходится основное время работы:

- Построение лапласиана по изображению
- Вычисление минимальной ненулевой собственной пары

Первое ядро

Основное время работы программы (больше 90% времени) занимает первое ядро. Вычисление для каждой вершины цветового различия с 8-ю соседями — большая вычислительная задача. Для ее ускорения были предприняты следующие меры:

1. Не вычислять одно и то же цветовое различие дважды: для каждого из соседних пикселей. Вместо этого была реализована другая логика заполнения лапласиана, позволяющая заполнять сразу два элемента лапласиана (лапласиан — симметричная матрица).
2. Сменить цветовое пространство с HSV на GrayScale. Это также позволило ускорить алгоритм, так как вычисление метрики в пространстве HSV требует значительно больше нетривиальных арифметических операций, чем в пространстве GrayScale (нужно всего одно вычитание).

Таким образом, получилось уменьшить время работы первого ядра с 90% времени работы программы, до 60-70%. Соответственно уменьшилось и время работы всей программы.

Алгоритмическая сложность этого ядра алгоритма есть $O(n)$, где $n = wh$: заполнение лапласиана порядка n требует $O(n)$ операций, так как именно столько ненулевых элементов содержится в лапласиане (для хранения лапласиана и вычисления операций используется разреженная матрица).

Второе ядро

Наиболее трудоемкая операция во втором ядре — это умножение разреженной матрицы на вектор. Эта операция требует $O(n)$ арифметических сложений и умножений с коэффицентом порядка 10 (столько умножений нужно для вычисления одного элемента в результирующем векторе, так как ненулевых элементов в каждой строке лапласиана не больше 10). Приемлемые результаты в обратном степнном методе достигаются всего за 3-4 итерации, на каждой из котрой вызывается метод минмальных невязок, в котором вектор сходится к решению с приемлемой точностью за 20-25 итераций в среднем. Таким образом, требуется около 60-100 умножений разреженной матрицы на вектор, то есть около 600-1000 арифметических умножений и сложений. Итого: сложность $O(n)$ с коэффициентом порядка 1000 арифметических операций умножения и сложения.

Время выполнения

В таблице ниже приведены результаты замеров времени выполнения программы на домашнем компьютере (`HP Laptop 15s-eq2028ur`) для изображений различных размеров $n = wh$:

n	time (sec)	n	time (sec)
1024	0.030	1.1e6	14.8
6800	0.095	1.9e6	20.5
20000	0.206	3.7e6	37.2
89800	0.950	8.8e6	80.5
171000	2.119	1.0e7	107.0
307000	3.234	1.9e7	201.4
640000	7.298	3.0e7	326.1

Таблица 1: Среднее время выполнения программы.

Таким образом, сложность действительно линейная.

Заключение

Построенный алгоритм справляется с задачей о выделении объекта на изображении с линейной от размера сложностью. В ходе работы также было выяснено, что с минимальными изменениями программы, алгоритм может выделять границы на изображении, причем справляется он с этим не хуже, а возможно и лучше.

Плюсы и минусы алгоритма

Среди **плюсов** алгоритма можно выделить следующее:

- хорошая детекция границ на изображении
- применимость алгоритма для решения различных задач обработки изображений
- линейная алгоритмическая сложность

Среди **минусов**:

- большое число арифметических операций
- большое время выполнения программы

Дальнейшее развитие

Алгоритм предусматривает существенное распараллеливание, и проект может найти развитие в:

1. распараллеливании процесса создания лапласиана
2. параллельном рекурсивном выделении подъобектов на отдельном объекте

Реализация этих возможностей является интересной задачей, которой автор планирует заняться в ближайшее время. Эта задача направлена на борьбу с минусами алгоритма, и решение этой задачи может подарить конкурентоспособный алгоритм детекции границ на изображении.

Список литературы

[1] M. Fielder, “Algebraic connectivity of graphs”, Czechoslovak Mathematical Journal, vol. 23, no. 2, pp. 298–305, 1973.

[2] M. Fielder, “A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory”, Czechoslovak Mathematical Journal, vol. 25, no. 4, pp. 619–633, 1975.

[3] Е.Е. Тыртышников “Методы численного анализа”, Москва, 2006.